Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph

Yiming Lin*
University of California, Irvine
yiminl18@uci.edu

Yeye He
Microsoft Research
yeyehe@microsoft.com

Surajit Chaudhuri
Microsoft Research
surajitc@microsoft.com

ABSTRACT

Business Intelligence (BI) is crucial in modern enterprises and billion-dollar business. Traditionally, technical experts like database administrators would manually prepare BI-models (e.g., in star or snowflake schemas) that join tables in data warehouses, before less-technical business users can run analytics using end-user dashboarding tools. However, the popularity of self-service BI (e.g., Tableau and Power-BI) in recent years creates a strong demand for less technical end-users to build BI-models themselves.

We develop an Auto-BI system that can accurately predict BI models given a set of input tables, using a principled graph-based optimization problem we propose called *k-Min-Cost-Arborescence* (k-MCA), which holistically considers both local join prediction and global schema-graph structures, leveraging a graph-theoretical structure called *arborescence*. While we prove k-MCA is intractable and inapproximate in general, we develop novel algorithms that can solve k-MCA optimally, which is shown to be efficient in practice with sub-second latency and can scale to the largest BI-models we encounter (with close to 100 tables).

Auto-BI is rigorously evaluated on a unique dataset with over 100K real BI models we harvested, as well as on 4 popular TPC benchmarks. It is shown to be both efficient and accurate, achieving over 0.9 F1-score on both real and synthetic benchmarks.

PVLDB Reference Format:
Yiming Lin, Yeye He, and Surajit Chaudhuri. Auto-BI: Automatically Build BI-Models
Leveraging Local Join Prediction and Global Schema Graph. PVLDB, 16(10): 2578 - 2590, 2023.
doi:10.14778/3603581.3603596

1 INTRODUCTION

Business Intelligence (BI) is increasingly important in modern enterprises for data-driven decision making, and has grown into a multi-billion dollar business [25]. In traditional BI settings, database administrators (DBAs) typically need to manually prepare BI-models (table schemas and join relationships) in data warehouses, so that less-technical business users can perform ad-hoc analysis using tools like dashboards [16].

In recent years, in a growing trend called "*self-service BI*" [26] that is popularized by vendors like Tableau [11] and Power-BI [9], less-technical business users are increasingly expected to set up and perform BI analysis themselves, without relying on DBAs or central IT. The goal is to democratize BI, so that business users can make agile data-driven decisions themselves, without depending on technical users.

Building BI models: still a pain point. At a high level, there are two main steps in BI project: (1) building BI models, and (2) performing ad-hoc analysis by querying against BI models. While querying BI-models was made simple by vendors like Tableau and Power-BI (through intuitive user interfaces and dashboards) [28, 38], the first step of building "BI-models", a prerequisite for ad-hoc analysis, remains a key pain point for non-technical users.

In the context of self-service BI tools like Tableau and Power-BI, "BI-modeling" refers to the process of preparing raw data and establishing relationships between tables, where a central task is to establish join relationships for a given set of input tables[1]. This closely relates to foreign-key (FK) detection [18, 30, 57] but works specifically in the context of BI, where the resulting schema graphs from the modeling step frequently correspond to structures known as star-schema and snowflake-schema studied in data warehouses [16], like shown in Figure 1.

While self-service BI tools also attempt to improve the usability of the BI-modeling step through better GUI (e.g., allowing users to specify join columns using drag-and-drop) [10, 12], building BI models remains a key pain point. This is because when faced with a large number of tables, even experienced technical users like DBAs can find the task of identifying all possible join relationships challenging and time-consuming. For less-technical enterprise users who are not familiar with concepts like fact/dimension tables, building BI models from scratch can be a daunting challenge.

Foreign-key detection: not yet sufficient. It would clearly be useful, if the join relationships in BI models can be automatically predicted on given input tables (without requiring users to specify them manually). Since joins in BI models are often primary-key (PK) foreign-key (FK) joins, existing FK detection algorithms [18, 30, 57] would seem to apply.

To study this systematically, we harvested over 100K real BI models built using self-service BI tools, from public sources like GitHub and search engines. For each BI model file, we programmatically extract the input tables used in the model, as well as the ground-truth join relationships specified by users, thus creating a real BI benchmark for large-scale evaluation for the first time (prior work mostly use synthetic benchmarks for evaluation instead).

Our large-scale evaluation using these real BI datasets suggests that existing FK-detection algorithms are still insufficient for the

*Work done at Microsoft.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603596

[1]We note that more advanced BI modeling can involve additional steps such as schema redesign and performance optimization, which is usually out of scope for non-technical users in self-service BI tools, and thus not considered in this work.

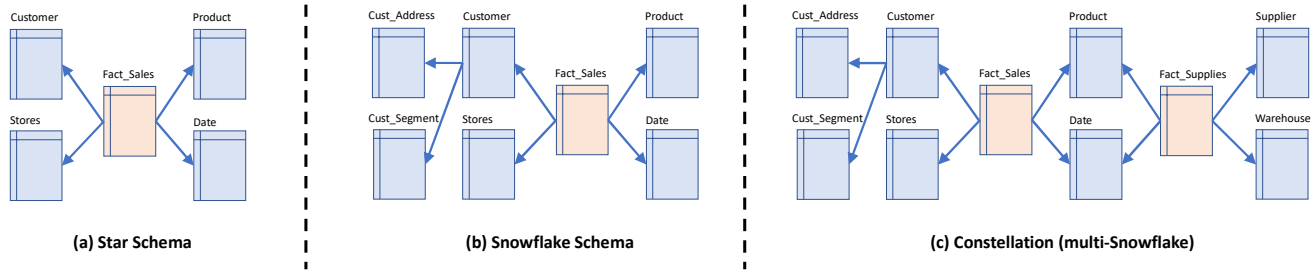


Figure 1: Types of common BI schemas. (a) Star schema, which can be seen as a simplified version of Snowflake. (b) Snowflake schema, where dimension tables like "Customers" can connect to additional dimension tables. (c) Constellation schema, which can be seen as multiple Snowflake schemas.

task, as they frequently produce incorrect results (with ~0.6 precision for all methods we tested). This is because real BI data in the wild tend to be considerably more challenging (e.g., column-headers can often be cryptic with generic names like "name" or "id", and column-values can often overlap accidentally). Crucially, most existing approaches are "local" in nature, that consider two tables at a time to make greedy local decisions, without a principled global optimization, thus unable to produce accurate predictions on challenging real BI test cases (Section 2).

Auto-BI: global optimization using schema graphs. A key insight we develop in Auto-BI, is that because we know users are finding join relationships specifically for building BI-models, and we know the typical structure that schema-graphs in BI-models should generally follow (e.g., variants of snowflake), this gives us a unique opportunity to leverage the *global graph structure* of the resulting schema-graph, to predict joins much more accurately.

We thus formulate the Auto-BI problem as a global optimization problem that holistically considers both the *local decision* of pairwise joinability, as well as the *global decision* of graph structures.

Specifically, we show that the snowflake schema popular in BI-modeling corresponds to a graph-theoretical concept called *Arborescence* [46]. Leveraging this structure, we formulate a new graph problem called *k-MCA (k-Minimum-Cost Arborescence)*, which finds the most probable k-snowflakes, by considering both local joinability and global graph structures, all using a precise probabilistic interpretation based on calibrated probabilities.

We prove that the new k-MCA problem and its variants are in general intractable (NP-hard) and inapproximable (EXP-APX-complete). Nevertheless, we develop novel algorithms based on branch-and-bound principles, which are surprisingly efficient on real test cases and can scale to the largest BI models we encounter, while still being provably optimal. Our extensive evaluations using real BI models and synthetic TPC benchmarks suggest that the proposed Auto-BI is substantially more accurate when compared to existing solutions in the literature.

Contributions. We make the following contributions:

- We are the first to harvest and leverage over 100K real BI models in the wild, for the problem of predicting BI models. This enables a data-driven algorithm design and makes it possible to rigorously evaluate different algorithms on real BI data.
- We are the first to exploit the snowflake-like schema structure in BI settings for our predictions, by mapping snowflakes to a less-known graph-theoretical concept called arboresence. We formulate a set of novel graph-based optimization problems that we call k-MCA, that have precise probabilistic foundations.
- We study the theoretical hardness of k-MCA variants, and propose efficient algorithms that are provably optimal. Extensive evaluations show that our algorithms are both effective (with over 0.9 F1 scores), and efficient (scales to the largest BI models we encounter and runs in a few seconds).

2 RELATED WORKS

BI and dashboarding tools. There are a wide variety of BI and dashboarding tools that aim to help users perform ad-hoc data analysis, with Tableau [11] and Power-BI [9] being the leading vendors [25]. These tools use visual drag-and-drop interfaces [38] (without requiring users to write SQL), and are particularly popular among non-technical users.

Foreign key detection. Foreign key (FK) detection is an important problem in database settings, with many influential methods developed in the literature [18, 30, 47, 57].

MC-FK [57] is a pioneering effort to detect FK using an EMD-based randomness metric for distribution similarity between two columns, which is more reliable to predict true relationships when unrelated key-columns that can frequently have overlapping ranges.

Fast-FK [18] develops an efficient method that selects FKs with the best pre-defined scores until all input tables are connected.

HoPF [30] improves upon prior work by considering not only FK-scores but also PK-scores, which are combined using a predefined scoring function, making this also a global method in spirit. The algorithm enumerates all PK/FK combinations and returns the combination that has the highest total score.

ML-FK [47] proposes an ML-based classifier to predict FK, trained on known FKs. As we will show analytically and experimentally, without principled global optimizations, ML classification alone is still local in nature and not sufficient to achieve high accuracy.

While FK detection methods are clearly related to our problem (we experimentally compare them with these methods), there are also a few key distinctions that we would like to highlight.

First, while FK detection targets general database settings, we focus on predicting joins in BI models, which gives us a unique opportunity to exploit the likely graph structure (e.g., snowflake) to make more accurate predictions, which is a unique direction not considered in prior work.

Second, unlike FK-detection methods that typically consider the canonical PK/FK (1:N) joins, in Auto-BI the types of joins we consider are more general, because real BI models in the wild frequently employ 1:1 joins as well as joins that are not perfectly 1:N, making the prediction problem more challenging.

Lastly, with the exception of [47], most prior FK detection methods primarily rely on hand-crafted scoring functions to make local join predictions (pairwise between two tables). In comparison, we leverage the BI models harvested to first predict local-joinability in a data-driven way, which is then combined into a principled global

optimization for optimal global decisions (at the graph-level for all tables). This also makes our technique different from prior work.

Detect inclusion dependency. Inclusion dependency (IND) is closely related to FK, with an influential body of work focusing on *efficiently* enumerating inclusion dependency (IND) in large databases [13, 33, 34, 39, 43, 49, 50, 52]. The focus on efficiency makes this line of work orthogonal to FK-detection, where the focus is to accurately predict meaningful FKs (from a large collection of IND candidates). Like prior FK-detection methods that employ efficient IND detection [30, 57], we also use IND-detection as a pre-processing step to enumerate possible FK candidates efficiently.

Complex non-equi joins. Beyond equi-joins, techniques to automatically detect and handle complex join relationships have also been studied, e.g., transformation-based join [42, 54, 59], fuzzy-join [17, 37, 51], search-join [36], semantic lookup-join [29], etc., which is an interesting area of future work in the context of BI.

3 PROBLEM STATEMENT

In this section, we first describe preliminaries and the real BI models we harvest, before introducing the Auto-BI problem.

3.1 Preliminary: BI models

Business Intelligence is closely related to topics like data warehousing and decision support system, and has been extensively studied. We provide a brief preliminary here and refer readers to surveys like [16, 32, 40] for more information.

Fact and dimension tables. In data warehousing and BI terminologies, tables involved in BI modeling can be categorized as two types: *fact tables* and *dimension tables* [32]. A *fact table* contains key metrics and measurements of business processes that one intends to analyze (e.g., the revenue of sales transactions). In addition, a fact table contains *foreign keys* that can reference multiple dimension tables, where each *dimension table* contains detailed information associated with the measurements from a unique facet (e.g., a "`Product`" dimension table contains details of products sold, whereas a "`Date`" dimension table has detailed day/month/year info of transactions, etc.). Figure 1 shows a few examples of the BI schemas, with fact and dimension tables in different colors.

Such a separation of fact/dimension tables has many benefits, such as storage/query efficiency, ease of maintenance, etc. [16, 32, 40]. The fact/dimension design is a de-facto standard in BI modeling.

Star, snowflake, and constellation schemas. In BI modeling, fact/dimension tables are frequently organized into what is known as *star/snowflake/constellation schemas* [32], like shown in Figure 1.

Star-schema refers to the cases where there is one fact table, whose foreign-key columns refer to primary-keys from one or more (non-hierarchical) dimension tables, as illustrated by Figure 1(a).

Snowflake-schema generalizes the star-schema, with dimension tables referring to each other in a hierarchical manner. For example, in Figure 1(b), the "`Customer`" dimension refers to a coarser-grained dimension "`Cust-Segment`". Similarly an "`Address`" dimension can refer to a coarser-grained "`City`", which in turn refers to "`Country`".

While there is only one fact table in star and snowflake schemas, *constellation-schema* generalizes to the cases with multiple fact tables, as shown in Figure 1(c).

We note that these three types of schemas are extensively studied in the literature [16, 32, 40] and widely adopted in practice.

3.2 Harvest Real BI Models

In order to understand real BI models created in the wild, and systematically evaluate the effectiveness of Auto-BI, we harvest over 100K real BI models from public sources that are created using a popular tool Power BI [9], whose model files have a suffix "`.pbix`".

We crawl these "`.pbix`" model files from two sources. The first is GitHub, where developers and data analysts upload their Power BI models for sharing and versioning. We crawl a total of 27K such model files from GitHub. As a second source, we go through the URLs crawled by a commercial search engine that can lead to "`.pbix`" model files. Most of these URLs are not crawled by the search engine so we crawl ourselves and obtain 86K model files.

Combining data from the two sources gives us a large collection of 100K+ real BI models, which cover diverse BI use cases, including financial reporting, inventory management, among many others.

These real BI models become a valuable dataset for rigorous evaluation – specifically, from each "`.pbix`" model file, we programmatically extract all tables used in the model, as well as the ground-truth BI model (join relationships) manually specified by users. This enables us to thoroughly evaluate different algorithms using real BI models in the wild, which turn out to be more challenging than synthetic TPC benchmarks used in prior work that tend to be clean and simple.

3.3 Problem Statement: Auto-BI

We now define the high-level Auto-BI problem as follows.

Definition 1. [Auto-BI]. Given a set of input tables $\mathbf{T} = \{T_1, T_2, \ldots, T_n\}$ used for BI modeling, where each table T_i consists of a list of columns $T_i = (c_{i1}, c_{i2}, \ldots, c_{im})$. Predict a desired BI model $M(\mathbf{T})$ that consists of a set of joins $M(\mathbf{T}) = \{J_{ij}, J_{pq}, \ldots\}$, where each join J_{ij} is a join between T_i and T_j, in the form of $J_{ij} = \big((c_{ik}, c_{il}, \ldots), (c_{jr}, c_{js}, \ldots)\big)$.

Note that the output $M(\mathbf{T})$ is restricted to equi-joins for now, which can be single-column joins, or multi-column joins in the form of $J_{ij} = \big((c_{ik}, c_{il}, \ldots), (c_{jr}, c_{js}, \ldots)\big)$. Also note that we only state the high-level problem so far, and will defer the exact graph-based formulation to Section 4.3.

A more general version of the problem goes beyond equi-joins in Definition 1 to handle complex forms of joins such as transformation-joins [59] and fuzzy-joins [37], which we will leave as future work.

4 AUTO-BI

We will start with an overview of our Auto-BI architecture, before discussing how we predict joins holistically on graphs.

4.1 Architecture Overview

Figure 2 shows the overall architecture of our Auto-BI system. It consists of an offline component and an online component, as depicted by the two boxes shown in the figure.

In the offline step, using real BI models we harvested and their ground-truth BI join models, we perform offline training to learn what we call "local join models" that can predict, for a given pair

CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning

HAOTIAN GAO, National University of Singapore, Singapore and NUSRI Chongqing, China
SHAOFENG CAI, National University of Singapore, Singapore
TIEN TUAN ANH DINH, Deakin University, Australia
ZHIYONG HUANG, National University of Singapore, Singapore and NUSRI Chongqing, China
BENG CHIN OOI, National University of Singapore, Singapore

Machine learning models are only as good as their training data. Simple models trained on well-chosen features extracted from the raw data often outperform complex models trained directly on the raw data. Data preparation pipelines, which clean and derive features from the data, are therefore important for machine learning applications. However, constructing such pipelines is a resource-intensive process that involves deep human expertise.

Our goal is to design an efficient framework for automatically finding high-quality data preparation pipelines. The main challenge is how to explore a large search space of pipeline components with the objective of computing features that maximize the performance of the downstream models. Existing solutions are limited in terms of feature quality, which results in low accuracies of the downstream models, while incurring significant runtime overhead. We present CtxPipe, a novel framework that addresses the limitations of previous works by leveraging contextual information to improve the pipeline construction process. Specifically, it uses pre-trained embedding models to capture the data semantics, which are then used to guide the selection of pipeline components. We implement CtxPipe with deep reinforcement learning and evaluate it against state-of-the-art automated pipeline construction solutions. Our comprehensive experiments demonstrate that CtxPipe outperforms all of the baselines in both model performance and runtime cost.

CCS Concepts: • **Information systems** → **Data analytics**; **Data cleaning**; • **Computing methodologies** → **Machine learning**.

Additional Key Words and Phrases: Data Analytics Pipeline, Data Preparation

ACM Reference Format:
Haotian Gao, Shaofeng Cai, Tien Tuan Anh Dinh, Zhiyong Huang, and Beng Chin Ooi. 2024. CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning. *Proc. ACM Manag. Data* 2, 6 (SIGMOD), Article 231 (December 2024), 27 pages. https://doi.org/10.1145/3698831

1 Introduction

The quality of input data determines the quality of Machine Learning (ML) models. For machine learning applications involving relational data, simple tree-based models trained with well-selected features derived from the raw data often have comparable, if not better model performance than that of state-of-the-art deep learning models trained directly on the raw data [19, 20, 40, 48]. The

Authors' Contact Information: Haotian Gao, National University of Singapore, Singapore, Singapore and NUSRI Chongqing, Chongqing, China, gaohaotian@comp.nus.edu.sg; Shaofeng Cai, National University of Singapore, Singapore, Singapore, shaofeng@comp.nus.edu.sg; Tien Tuan Anh Dinh, Deakin University, Melbourne, Victoria, Australia, anh.dinh@deakin.edu.au; Zhiyong Huang, National University of Singapore, Singapore, Singapore and NUSRI Chongqing, Chongqing, China, huangzy@comp.nus.edu.sg; Beng Chin Ooi, National University of Singapore, Singapore, Singapore, ooibc@comp.nus.edu.sg.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).
ACM 2836-6573/2024/12-ART231
https://doi.org/10.1145/3698831

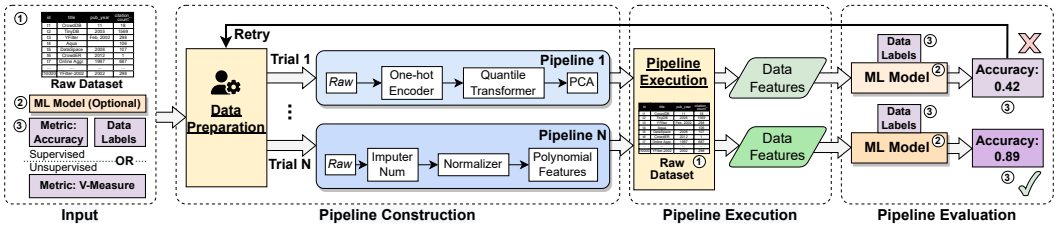


Fig. 1. Constructing a data preparation pipeline to maximize the performance of the generated data features.

tree-based models are not only more efficient to train but also have better interpretability [39, 46]. Extracting high-quality features from raw data requires a *data preparation pipeline*, which comprises multiple components that clean and transform raw data into features [1, 33]. These components either come from the ML frameworks, or are defined by the user.

The process of building a data preparation pipeline that generates high-quality features, or *data preparation pipeline construction*, is illustrated in Figure 1. It is performed by a data scientist, taking as input the raw dataset, an ML model, and its corresponding metric for evaluation. For supervised learning, the data labels are also provided. The process is manual and iterative. At each iteration, the scientist constructs a pipeline, executes it to derive a set of features, trains the model, and observes the model performance. The final output is the pipeline that achieves the best model performance.

Our goal is to design an efficient framework for automatically finding high-quality data preparation pipelines, that is, for automated data preparation pipeline construction. The main challenge is how to efficiently explore a large search space of pipeline components. In particular, the search space of possible component combinations grows exponentially with the number of possible components for different cleaning and transformation tasks [49]. Furthermore, the time cost of evaluating the pipelines is significant, because for each pipeline this cost includes the execution of the pipeline, as well as the model training and testing. With distribution shifts [32, 39, 47] requiring periodic pipeline rebuilding, this overhead accumulates and becomes significant over time.

Existing works on automated pipeline construction fall short of achieving our goal [8, 16, 21, 24, 26, 28–30, 33, 42, 46, 49, 54, 60]. Different techniques, e.g., genetic programming [42, 49], Bayesian optimization [16, 46, 54], reinforcement learning [8, 11, 21, 46], and differential formulation [24, 33, 60] are used to select and permute pipeline components. However, the search space still grows exponentially, resulting in significant runtime overhead. One reason for the large search space is that these works only use table schema and other statistical information to guide the exploration.

Our insight is that we can leverage *contextual information* to significantly reduce the search space. Intuitively, by capturing the *domain expertise* that data scientists use during the manual construction of data preparation pipelines, we can perform a more guided, efficient search. Such domain knowledge from the data, or the contextual information, can be derived from multiple sources (Figure 2). For example, the domain of the datasets (such as healthcare, finance, e-commerce, etc.) can be inferred from the column names and concrete string values in the table. The relationship across columns can be derived from columns with similar names. The high-level semantics of statistical information can also be derived from the type and range of numerical values in the columns. These column semantics, if captured automatically, could enable a more fine-grained separation of dataset scenarios to the pipeline construction solution, thereby improving its predictive performance.

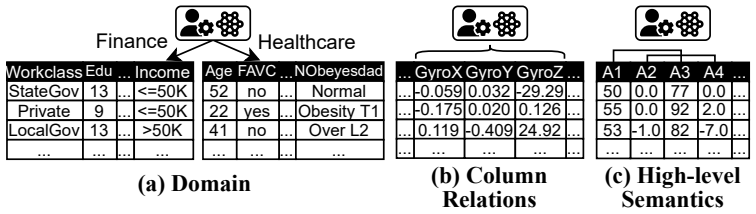


Fig. 2. Examples of contextual information.

We present CtxPipe[1], an efficient framework for automated data preparation pipeline construction. It addresses the limitations of previous works by leveraging contextual information from the data to improve the pipeline search. More specifically, CtxPipe employs state-of-the-art pre-trained embedding models to capture the semantics of raw and transformed data, then uses them to dynamically manipulate the training and inference of a pipeline construction model via a *gated context plug-in* module. CtxPipe can identify the pipeline that produces high-quality features, because pipeline search is guided within a specific context. In contrast, previous works use only table statistics to guide the search, which means that two tables with similar statistics but different semantics may produce similar pipelines and the corresponding features. CtxPipe implements the context plug-in based on deep reinforcement learning [11, 21]. It further improves the module robustness by introducing an optimized agent optimization algorithm called *Open-Closed-Gated (OCG) experience replay* that performs controlled backpropagations under marginal plug-in setups.

The most similar system of CtxPipe is HAIPipe [11], which leverages pre-built pipelines to complement the automatically generated pipelines. However, such pre-built pipelines are created manually, making the entire process less automated. In contrast, our system integrates external knowledge while enabling a fully automated pipeline construction workflow. To the best of our knowledge, we are the first to design and implement a fully automated, context-aware solution for data preparation pipeline construction.

In summary, we make the following contributions:

- We present CtxPipe, a novel and efficient framework for automated data preparation pipeline construction. It makes use of embedding models to automatically extract context into vectors, which are then integrated into the pipeline construction algorithm via a *gated context plug-in* module.
- We propose an agent optimization strategy called *Open-Closed-Gated (OCG) experience replay* that performs effective and stable joint optimization on the main deep reinforcement learning network and the context plug-in.
- We compare CtxPipe against state-of-the-art solutions in terms of the quality of transformed data on 18 commonly used datasets. Our extensive experimental results show that CtxPipe outperforms the baselines in all evaluation metrics, and achieves up to 170.99x speedup in pipeline construction compared to the second-best baseline.

The remainder of the paper is structured as follows. Section 2 reviews the related work. Section 3 presents the preliminaries and the problem definition. Section 4 overviews CtxPipe, focusing on its pipeline construction agent. Section 5 discusses the context plug-in in detail, including its context integration mechanism and agent optimization. Section 6 discusses the implementation, followed by the evaluation results in Section 7. Section 8 concludes the paper.

[1]The source code of CtxPipe is available at: https://github.com/ctxpipe.

FEATPILOT: Automatic Feature Augmentation on Tabular Data

Jiaming Liang*
University of Pennsylvania
liangjm@seas.upenn.edu

Chuan Lei, Xiao Qin, Jiani Zhang, Asterios Katsifodimos,
Christos Faloutsos, Huzefa Rangwala
Amazon Web Services
chuanlei, drxqin, zhajiani, akatsifo, faloutso, rhuzefa@amazon.com

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00163

***Abstract*—Tabular data within enterprises or open data repositories provide a huge opportunity for feature augmentation. Using these data sources to augment training data often boosts model performance, which is crucial in data-centric AutoML systems. Recent works on automatic feature augmentation have limited capabilities in utilizing useful features that cannot be joined with the base table without connecting through intermediate tables. We present FEATPILOT, a novel framework that explores and integrates high-quality features in tabular data for ML models. FEATPILOT evaluates a candidate feature from two aspects: (1) the efficacy of a join path connecting the feature to the base table and (2) the intrinsic value of a feature towards an ML task. FEATPILOT efficiently identifies high-quality features and their optimized join paths to augment the base table. Our experimental results show that FEATPILOT achieves up to a 10.27% improvement in ML model performance compared to state-of-the-art solutions across six public datasets.**

I. INTRODUCTION

Feature augmentation on tabular data is crucial for improving machine learning (ML) model performance [1]–[5]. With additional features, models can capture more complex patterns and relationships within the data, leading to better prediction and generalization results. By exploring and integrating features [6], [7] from open tabular data repositories (e.g., Google Dataset Search[1], Kaggle[2], and OpenML[3]), ML models [8] can avoid overfitting [9] and improve performance on unseen data [3]–[5].

Given a base table with an ML task defined on it and a set of candidate tables that can be joined with the base table via one-hop or multi-hop paths, feature augmentation is the task of selecting feature columns from the candidate tables that can maximize the performance of the ML task. This process typically involves identifying useful features that can reveal additional insights or patterns for the given ML task, and determining the optimal paths to join these candidate features with the base table to enrich the initial training data.

Motivating Example. An ML engineer is developing a classification model to predict whether a product should be recommended or not. The base table `product` contains only a limited set of features for building a high-quality model (shown in Fig. 1). Fortunately, useful features exist in related tables such as `address` in the `customer` table. To augment the base table, the ML engineer needs to find a high-quality join path to augment the base table. Among three join paths in Fig. 1, `customer` ⋈ `order` ⋈ `product` (indicated by the thick red line) provides more feature instances (i.e., `address`) compared to paths involving the `rating` or `review` table, as customers may not provide ratings or reviews for every purchased product. As a result, the ML engineer can exploit this augmented table to improve the classification model.

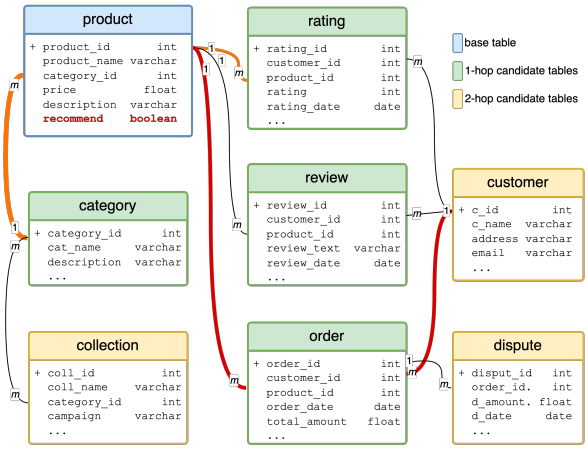


Fig. 1: Example of feature augmentation. The base table, `product`, is in blue. The task is to predict the values in the Boolean type column 'recommend'. The tables in green and yellow are 1-hop and 2-hop joinable with the base table.

Challenges. In this paper, we address the following three key challenges in feature augmentation.

(1) *Multiple Join Paths Selection.* In real-world open data settings, a candidate feature can be integrated with the base table via multiple join paths, resulting in different integration quality as explained in the motivating example. Hence, it is critical to identify the optimal join path to achieve the full potential of a candidate feature.

(2) *Expensive Join Operations.* To find out the actual benefit of augmenting the base table with a candidate feature, expensive joins need to be executed, which is computationally expensive and time consuming, especially when a long join path involves multiple joins. Candidate features in open data repositories often have similar join paths. Therefore, sharing join operations is crucial for efficient feature augmentation.

*The work was done when Jiaming Ling was with Amazon Web Services.
[1]https://datasetsearch.research.google.com/
[2]https://www.kaggle.com/datasets
[3]https://www.openml.org/

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00163

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

(3) *Search Complexity.* The search space of all candidate tables and their join paths is vast and intractable, making it prohibitively expensive to find the optimal set of candidate tables. To address this challenge, we need effective strategies to reduce the search space while preserving optimality.

State-of-the-art Approaches. Prior approaches [3], [10] augment the base table by joining other tables using either their relevancy to the base table or the improvement of the candidate features bring to the ML models. Recently, ALITE [11] and DIALITE [12] are introduced to integrate tables based on joinability or unionability, which can improve the quality of downstream ML models. However, these approaches assume that the features and their respective relevance to the base table are specified. Hence, they fail to leverage all possible candidate features and consequently do not achieve optimal model performance. Reinforcement learning (RL) [13], [14] has emerged as an effective approach to balance between feature exploration and exploitation. Several studies [4], [5], [15] leverage RL techniques such as multi-armed bandits and Deep Q-Networks (DQNs) to explore the large search space of candidate tables. Although these RL-based methods have achieved promising results, they still face critical challenges. Specifically, METAM [5] is limited to directly joinable candidate tables and AutoFeature [4] does not provide fine-grained optimization on integration quality. Moreover, RL often requires extensive training data and tends to overfit, reducing its effectiveness in real-world applications.

Proposed Solution. To cope with the above mentioned challenges, we propose FEATPILOT, an efficient and effective framework for automatic feature augmentation. Given a base table, a set of candidate tables and an ML task, FEATPILOT first selects a subset of candidate tables using a clustering algorithm based on table characteristics. FEATPILOT further chooses representative join paths from these candidate tables to the base table. This allows FEATPILOT to discover the actual performance gain led by the candidate tables following the join paths (*Challenge 1*). Consequently, FEATPILOT learns an integration quality model and a feature importance estimator to calculate the performance gain of any unexplored candidate tables and their respective join paths (*Challenge 2*). Lastly, FEATPILOT's join-path search algorithm prunes the search space based on both integration quality and feature importance, and finds the optimized integration paths for the selected candidate tables in polynomial time (*Challenge 3*).

Contributions. Our main contributions are the following.

(1) We introduce FEATPILOT, a novel feature augmentation framework for ML tasks over tabular data repositories, by decomposing the intractable problem of feature augmentation into feature importance and integration quality.

(2) We design an effective feature exploration module consisting of a clustering-based feature importance estimation and a learning-based integration quality model.

(3) We propose an efficient feature path search algorithm that exploits both estimated feature importance and integration quality to identify high-quality features and their optimized join paths for integration.

(4) Our experiments show that FEATPILOT achieves up to a 10.27% improvement in ML model performance compared to state-of-the-art solutions across six public datasets.

Reproducibility. The source code of FEATPILOT is available at `https://anonymous.4open.science/r/FeaturePilot-C308`.

II. Preliminaries and System Overview

A. Basic Notions

Definition 1 (Base Table): The base table, T_{base} = $\{c_1, c_2, \ldots, c_n, c_l\}$, refers to the original table that includes the initial set of features and the label column before any augmentation. Each c_i ($i \in [1, n]$) represents a column and c_l denotes the label column, which stores the prediction target for each row.

Definition 2 (Candidate Tables): The candidate tables, $\mathcal{T}$ = $\{T_1, T_2, \ldots, T_m\}$, are m joinable tables that can be used to augment a base table T_{base}. Each table is denoted by T_j = $\{f_{j,1}, f_{j,2}, \ldots, f_{j,k}\}$ ($j \in [1, m]$), where $f_{j,k}$ denotes the k-th column (a.k.a feature)[4] in table T_j.

Definition 3 (Join Path): Given a set of tables $T_1, T_2, \ldots, T_n$, a join path P can be defined as a sequence of join operations between pairs of tables. A join operation between any two tables T_i and T_j can be denoted as $T_i \bowtie T_j$. The join path P can be represented as: $P = T_1 \bowtie T_2 \bowtie T_3 \ldots \bowtie T_n$.

Without loss of generality, we allow different types of join operators between T_{base} and $T_j \in \mathcal{T}$, including inner join, left outer join, right outer join, etc.

Definition 4 (Augmented Table): The augmented table $T_{aug} = T_{base} \bowtie_P T_j$ is the result of joining the base table T_{base} and a candidate table T_j following a join path P.

Note that there can be multiple join paths between the base table and a candidate table. Hence the augmented table could be different depending on the chosen join path.

Definition 5 (Machine Learning Task): A machine learning (ML) task is defined as a function f that maps an input space $\mathcal{X}$ to an output space $\mathcal{Y}$. Given a dataset $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$, the objective is to learn the function f that best approximates the true underlying relationship from the inputs to the outputs.

Definition 6 (Utility Gain): Given a ML task, the utility score, $US(T)$, is defined as the performance metric (e.g., accuracy, precision, recall, etc.) of the task on a table T achieved by a specific ML model[5]. Let $US(T_{base})$ be the performance metric on the base table, and $US(T_{aug}) = US(T_{base} \bowtie_P T_j)$ be the performance metric on the augmented table. The utility gain UG is defined as the improvement in the performance metric: $UG = US(T_{aug}) - US(T_{base})$.

Problem Statement. Given a base table T_{base}, a set of candidate tables $\mathcal{T}$, and a ML task, our goal is to select features from these candidate tables $\mathcal{T}$ and to identify the best join paths to augment the base table T_{base} with the selected

[4]We use two terms, column and feature, alternately in this paper.

[5]$US(T)$ uses the negation of a metric value when smaller values are preferred (e.g., mean squared error (MSE), mean absolute error (MAE), etc.).

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

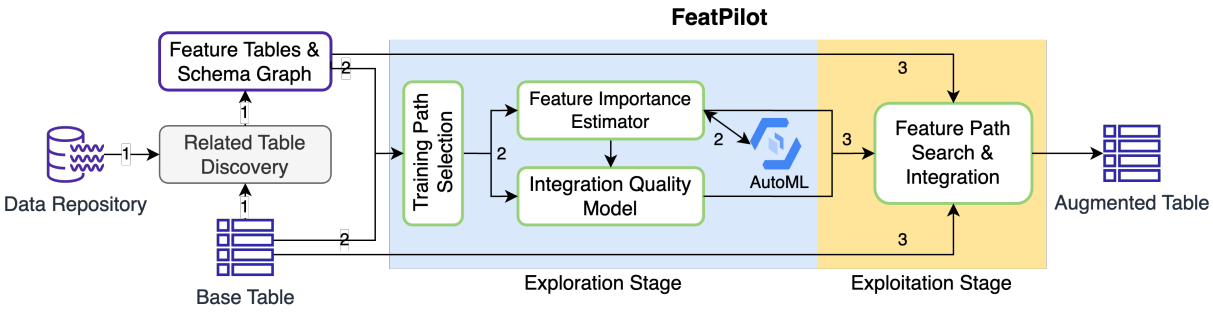


Fig. 2: FEATPILOT overview.

features, such that the utility gain UG of the ML task on T_{aug} is maximized. Formally,

$$\max_{\mathcal{S}\subseteq\mathcal{T},\mathcal{P}} UG = US(T_{aug}) - US(T_{base}) \quad \text{subject to } T_{aug} = T_{base} \bowtie_{\mathcal{P}} \mathcal{S} \tag{1}$$

where $\mathcal{S} \subseteq \mathcal{T}$ is the subset of selected candidate tables from $\mathcal{T}$, $\mathcal{P}$ is the set of join paths used to augment T_{base} with $\mathcal{S}$, and T_{aug} is the augmented table obtained by joining T_{base} with the selected tables $\mathcal{S}$ following the join paths $\mathcal{P}$.

Note that we treat the ML model (e.g., linear regression, deep neural network, XGBoost etc.) and the corresponding task as a black box such that the user can leverage our framework to augment features regardless of any particular model, and the model parameters can be updated iteratively.

The complexity of feature augmentation problem generally increases exponentially with the number of candidate features and all possible join paths among the base and candidate tables. The combination of these two exponential search spaces makes the problem NP-hard. Even with various heuristic and optimization techniques [16], [17] to reduce the complexity, the cost of feature augmentation can be still expensive.

B. FEATPILOT System Overview

For a given ML task, FEATPILOT takes input as a base table with a prediction task on the target column and a tabular data repository consisting of candidate tables with features. As depicted in Fig. 2, FEATPILOT leverages data discovery systems such as Aurum [18] or NYU Auctus [19] to identify the candidate tables within k-hop joins to the base tables. FEATPILOT then triggers its utility gain model in two stages, i.e., exploration and exploitation.

FEATPILOT's exploration stage tackles *Challenge 1* and *Challenge 2* mentioned in Section I. It takes the following four steps. (1) FEATPILOT groups the identified features into clusters based on their representations (embeddings, data distributions, data types, etc.). (2) FEATPILOT employs a path selection strategy to choose a set of features and candidate join paths. (3) FEATPILOT then follows the selected join paths to integrate the candidate features into the base table and runs AutoML (e.g., AutoGluon[6]) to compute the utility gain over the model trained on the base table. (4) Using the data collected in (3), FEATPILOT trains an integration quality model and assigns the actual feature importance to the selected features in the clusters. The importance of the other features can be estimated based on their relative distance to the ones with the actual importance assigned.

FEATPILOT's exploitation stage address *Challenge 3* discussed in Section I. It leverages both the feature importance estimator and integration quality model to conduct an efficient and effective search over all candidate features and their associated join paths. It iteratively selects the most important feature and uses the integration quality model to select the join path with the best integration quality until a predefined number of features is reached. FEATPILOT then follows the selected join paths to integrate the selected features with the base table and generates the final augmented table for the given ML task. Finally, AutoML solutions can train, select and deploy the best performing ML model on the augmented table [20].

III. UTILITY GAIN MODELING

The problem of *maximizing utility gain* for a given ML task is intractable. Hence we propose to decompose this problem into two sub-problems, namely maximizing feature importance – how useful is a feature to the ML model, and maximizing integration quality – how many data instances in the base table can be augmented by a feature.

Feature Importance. To define the *feature importance*, we first introduce the concept of a *virtual table*. Let T_{aug} denote a resulting augmented table from a base table T_{base} and a candidate table T_c. The virtual table T_{virt} refers to the ideal case where there exists a join path P such that every instance in T_{base} can be augmented by a feature instance from T_c via this P (i.e., iPhone 15 with a rating of 5, Pixel 8 with a rating of 4.5, and Galaxy S23 with a rating of 4 shown in Fig. 3b). Note that a virtual table is specific to a base table and an individual candidate feature.

Note that, for ease of illustration, we assume a 1:1 join relationship in this example. In practice, FEATPILOT supports 1:1, 1:m, and m:m join relationships between the base table T_{base} and a candidate table T_j. To handle these scenarios, FEATPILOT employs commonly used methods based on feature data types, including aggregation (e.g., count, mean, min/max, etc. for numerical features), pooling on feature embeddings (for textual features), one-hot encoding (for categorical features).

[6] https://auto.gluon.ai/

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation

SIBEI CHEN, Renmin University of China, China
NAN TANG, QCRI / HKUST (GZ), Qatar / China
JU FAN*, Renmin University of China, China
XUEMI YAN, Renmin University of China, China
CHENGLIANG CHAI, Beijing Institute of Technology, China
GUOLIANG LI, Tsinghua University, China
XIAOYONG DU, Renmin University of China, China

Data preparation is crucial in achieving optimized results for machine learning (ML). However, having a good data preparation pipeline is highly non-trivial for ML practitioners, which is not only domain-specific, but also dataset-specific. There are two common practices. Human-generated pipelines (**HI-pipelines**) typically use a wide range of *any* operations or libraries but are highly experience- and heuristic-based. In contrast, machine-generated pipelines (**AI-pipelines**), *a.k.a.* AutoML, often adopt a predefined set of sophisticated operations and are search-based and optimized. These two common practices are mutually complementary. In this paper, we study a new problem that, given an HI-pipeline and an AI-pipeline for the same ML task, can we combine them to get a new pipeline (**HAI-pipeline**) that is better than the provided HI-pipeline and AI-pipeline? We propose HAIPipe, a framework to address the problem, which adopts an enumeration-sampling strategy to carefully select the best performing combined pipeline. We also introduce a reinforcement learning (RL) based approach to search an optimized AI-pipeline. Extensive experiments using 1400+ real-world HI-pipelines (Jupyter notebooks from Kaggle) verify that HAIPipe can significantly outperform the approaches using either HI-pipelines or AI-pipelines alone.

CCS Concepts: • **Information systems → Data analytics**.

Additional Key Words and Phrases: pipeline generation, data preparation, reinforcement learning

ACM Reference Format:
Sibei Chen, Nan Tang, Ju Fan, Xuemi Yan, Chengliang Chai, Guoliang Li, and Xiaoyong Du. 2023. HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation. *Proc. ACM Manag. Data* 1, 1, Article 91 (May 2023), 26 pages. https://doi.org/10.1145/3588945

1 INTRODUCTION

Data preparation (data prep for short) is the process of turning raw data into a format that is ready for use by data science or ML tasks. This process is often implemented by a series of steps (*a.k.a.* a

*Ju Fan is the corresponding author.

Authors' addresses: Sibei Chen, Renmin University of China, Beijing, China, sibei@ruc.edu.cn; Nan Tang, QCRI / HKUST (GZ), Doha / Guangzhou, Qatar / China, ntang@hbku.edu.qa; Ju Fan, Renmin University of China, Beijing, China, fanj@ruc.edu.cn; Xuemi Yan, Renmin University of China, Beijing, China, yxmlpzt@ruc.edu.cn; Chengliang Chai, Beijing Institute of Technology, Beijing, China, ccl@bit.edu.cn; Guoliang Li, Tsinghua University, Beijing, China, liguoliang@tsinghua.edu.cn; Xiaoyong Du, Renmin University of China, Beijing, China, duyong@ruc.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART91 $15.00
https://doi.org/10.1145/3588945

pipeline) that the data moves through from one step to its subsequent steps. However, data prep is also known to be very hard and time-consuming [26].

Traditional data prep pipelines are human orchestrated, which are abbreviated as HI-pipelines. The pros of HI-pipelines are that users can use "any" operation and can easily inject domain knowledge (*e.g.,* "?" means a missing value for occupation). The cons are that users typically orchestrate pipelines based on their experience, which are heuristic and are rarely optimized, *e.g.,* humans never manually try all different combinations and find the best one.

Recent advances in the ML community have extended AutoML from hyper-parameter tuning and neural architecture search to an attempt that tries to automate the process of data prep for ML (*e.g.,* Auto-SKLearn [9] and DeepLine [12]) – these machine-generated pipelines are called AI-pipelines in this paper. The pros of AI-pipelines are that they use a constrained set of operations, for which they can find the best pipeline through exhaustively evaluating different pipelines within a pre-defined search space. The cons of AI-pipelines are that they lack domain knowledge.

Our **key observation** is that HI-pipelines and AI-pipelines are naturally complementary. That is, the pros of HI-pipelines (*i.e.,* using a wider range of operations based on domain knowledge) are the cons of AI-pipelines, and the cons of HI-pipelines (*i.e.,* experience-based and heuristic-based) are the pros of AI-pipelines (*i.e.,* highly optimized). Therefore, intuitively, HI-pipelines and AI-pipelines should be combined, so as to get the best of both worlds.

In this paper, we study a new problem: Given an HI-pipeline and an AI-pipeline for an ML data prep task, can we generate a better data prep pipeline (*i.e.,* an HAI-pipeline) by combining them?

Example 1. *Consider Figure 1. After loading the dataset (lines 1–4), an HI-pipeline consists of two components h_1 (lines 5-8,* i.e., *dropping rows with missing values) and h_2 (lines 24-28,* i.e., *scaling numerical attributes). Then, we use a machine-generated AI-pipeline that consists of three components a_1 (lines 10–15,* i.e., *encoding categorical attributes), a_2 (lines 17–22,* i.e., *generating interaction features) and a_3 (lines 30–35,* i.e., *removing useless features). A combination of these two pipelines is given in Figure 1, which outperforms both the HI-pipeline and the AI-pipeline.*

To address the problem, we propose HAIPipe, a framework that combines an HI-pipeline with an AI-pipeline for a specific ML task.

There are **two challenges**. (1) How to combine an HI-pipeline and an AI-pipeline? (2) How to obtain an optimized AI-pipeline from various possible combinations?

For Challenge (1), we devise an *enumeration-sampling* approach that first enumerates possible HAI-pipelines, and then employs an active learning strategy to sample a limited number of HAI-pipelines for evaluation, so as to find the one with the highest performance. For Challenge (2), there are a variety of operation families (*e.g.,* `Imputer` and `Scaler`). Within each family, there are many operations (*e.g.,* `MinMaxScaler` and `StandardScaler`). Thus, there exist many possible ways of selecting operations to form an AI-pipeline, which are highly dataset-specific and task-specific. We introduce a reinforcement learning (RL) based approach that learns how to select operations through offline training.

Contributions. Our contributions are summarized as follows.

(1) We formally define data prep pipelines, and introduce HI-pipelines and AI-pipelines (Section 2).

(2) We propose a framework, namely HAIPipe, that combines HI-pipelines and AI-pipelines for data prep (Section 3). We devise an enumeration-sampling strategy that effectively and efficiently combines HI-pipelines and AI-pipelines (Section 4).

(3) We present an RL-based AI-pipeline algorithm that pays particular attention on optimizing data prep operations (Section 5).

(4) We conduct extensive experiments to show that HAIPipe can significantly outperform both the

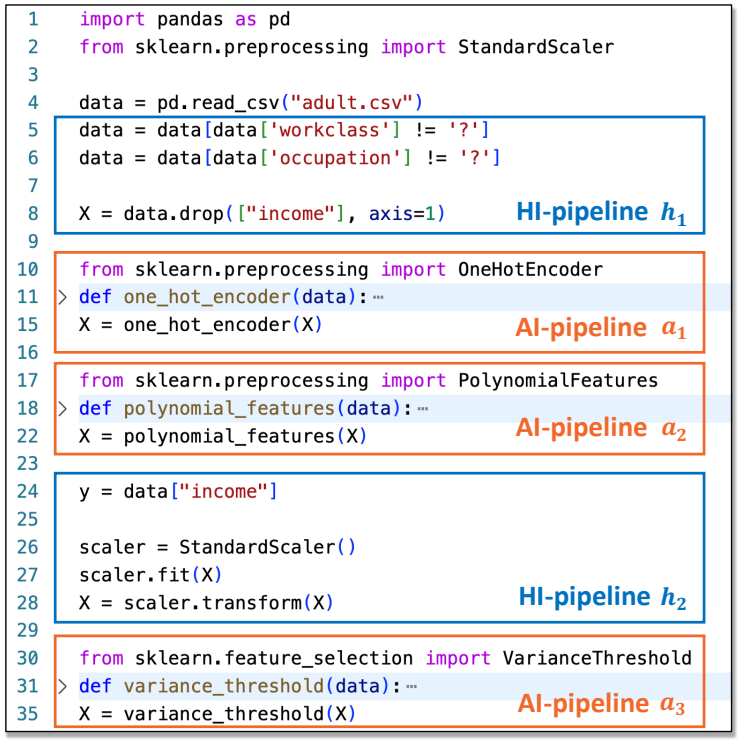


Fig. 1. An example of combining an HI-pipeline and an AI-pipeline. Blue blocks: human-written scripts. Orange blocks: machine-generated scripts.

HI-pipelines and AI-pipelines (Section 6). For reproducibility, we make code and datasets in our experiments public on GitHub[1].

2 DATA PREP PIPELINES

Let $\mathcal{T}(D, M)$ be an ML task with an ML model M and training data D. Without loss of generality, this paper considers classification models for ease of presentation. Our techniques can be easily extended to other supervised ML models. Practically, finding the "best" model M often requires a sequence of *data prep steps* to obtain the "best" data, including imputation, scaling, etc., which are naturally formalized as *operations* as below.

Data prep operations. A *data prep operation* (or operation for short), which is denoted as o, encapsulates a specific functionality that transforms a dataset D into another dataset D', *i.e.,* $D' = o(D)$. Let Σ denote the universe of operations for our task $\mathcal{T}$, which contains a set of diversified *families* of data prep functionalities. Among them, some rely on well-developed algorithms in known Python libraries, such as Scikit-Learn [27]. In this paper, we consider the following families of data prep algorithms implemented in Scikit-Learn (*i.e.,* `sklearn`), likewise in some existing studies [12, 35].

- `Imputer`: a data cleaning step that fills in missing values in dataset D, *e.g.,* `impute.SimpleImputer`.

[1]https://github.com/ruc-datalab/Haipipe